

Tutorial - Semana 7, Encontro 1

Introdução

Ao final da Semana 6, o projeto tem `GameManager` / `SaveManager` (Autoload), o contrato `Interactable`, Signals, e a classe `ItemData` (Resource + Enum) aplicada a um conjunto próprio de itens coletáveis por grupo. Todo esse estado, porém, vive apenas em memória durante a execução: o `SaveManager` mantém dados entre trocas de cena, mas ao fechar o jogo tudo se perde. Este encontro resolve esse problema em cinco etapas: constrói `SaveData` (Resource) + `FileAccess` e um `SaveComponent` que centraliza a leitura/escrita; constrói as Scenes `Pickup` e `Chest`, os primeiros objetos interativos que entregam um `ItemData` ao jogo; constrói a Scene `Checkpoint`, que aciona a gravação reutilizando o mesmo contrato `Interactable` já usado por portas e alavancas desde a Semana 5; e fecha o ciclo carregando o save ao iniciar o nível e escolhendo o ponto de spawn do Player (`PlayerStart` ou último `Checkpoint`) via o `GameManager.spawn_player()` da Semana 4.

Objetivos da semana

- Diferenciar persistência entre cenas (`SaveManager`, Semana 4) de persistência entre sessões (`SaveData` + `FileAccess`).
- Construir `SaveData` como Resource customizado e um `SaveComponent` que centraliza sua leitura/escrita.
- Construir as Scenes `Pickup` e `Chest`, aplicando o `ItemData` (Semana 6) a objetos coletáveis via contrato `Interactable`.
- Construir a Scene `Checkpoint`, reutilizando o contrato `Interactable`.
- Fechar o ciclo do save: carregar o `SaveData` ao iniciar o nível e fazer `GameManager.spawn_player()` escolher entre o `PlayerStart` e o último `Checkpoint` alcançado.

Resultado esperado ao final da semana

Este tutorial cobre apenas o **Encontro 1**: ao final dele, cada grupo tem um `SaveData` funcional salvando e recuperando progresso entre sessões, ao menos um `Pickup` e um `Chest` concedendo itens do conjunto do grupo, uma Scene `Checkpoint` que aciona essa gravação via o contrato `Interactable`, e o ciclo fechado — ao reabrir o jogo, o Player nasce no último checkpoint alcançado. O Encontro 2 integra este resultado ao restante dos sistemas do Módulo 2.

Pré-requisitos

- `GameManager` (com `spawn_player()` e o `PlayerStart` no nível) e `SaveManager` (Autoload) da Semana 4, funcionando.
 - Contrato `Interactable`, `Signals` e ao menos um objeto interativo (Door ou Lever) da Semana 5.
 - Classe `ItemData` (Resource + Enum) e o conjunto próprio de itens coletáveis de cada grupo, da Semana 6.
-

Antes de começar

O que o estudante deverá possuir antes desta semana

- O projeto do Vertical Slice herdado da Semana 6, com `ItemData`, Enum e as instâncias `.tres` de itens já testadas.

Arquivos necessários

- Nenhum arquivo externo adicional.

Assets utilizados

- Nenhum asset novo. Este encontro é inteiramente de scripting e configuração de Scene.

Projeto esperado

- Projeto aberto no Godot 4.7, com `GameManager`, `SaveManager`, contrato `Interactable` e `ItemData` já funcionais.

Imagem sugerida

Objetivo: contrastar visualmente persistência entre cenas e persistência entre sessões.
Enquadramento: diagrama simples de duas colunas. Elementos importantes: à esquerda, "SaveManager (Autoload)" com uma seta circular entre duas cenas do mesmo jogo em execução; à direita, "SaveData + FileAccess" com uma seta entre um ícone de jogo em execução e um ícone de disco/pasta `user://`, e outra seta de volta ao abrir o jogo

novamente. Destaque: a pasta `user://` como o elemento que sobrevive ao fechamento do processo. Legenda sugerida: "SaveManager mantém estado entre cenas; SaveData + FileAccess mantém estado entre sessões, gravando em disco."

Parte 1 — SaveData: o Resource que representa o progresso salvo

Objetivo

Criar a classe `SaveData` como Resource customizado, contendo os dados de progresso a persistir.

Conceito

Toda engine com sessões de jogo separadas no tempo — o jogador desliga e volta no dia seguinte — precisa de um mecanismo que transforme o estado que existe apenas na memória do processo em algo gravado em disco, e o processo inverso ao reabrir o jogo. Isso é diferente do que o `SaveManager` já resolve: o `SaveManager` (Autoload) mantém uma variável viva enquanto o jogo está rodando, mesmo trocando de cena; mas se o processo do jogo é encerrado, essa variável desaparece com ele.

O Godot resolve a persistência em disco com Resources serializáveis. Assim como `ItemData` (Semana 6) é um Resource que representa dados de design, `SaveData` é um Resource que representa dados de progresso — a diferença é que `SaveData` não vive apenas como arquivo `.tres` dentro do projeto (`res://`), mas é gravado e lido dinamicamente na pasta de dados do usuário (`user://`), isolada do próprio projeto e disponível mesmo em um build exportado.

Passo a passo

1. No FileSystem Dock, dentro de `scripts/resources/`, crie um novo script chamado `save_data.gd`.
2. No topo do script, declare a classe e a herança de `Resource`:

```
class_name SaveData
extends Resource
```

3. Adicione um campo exportado para os itens coletados, como uma lista de identificadores:

```
@export var itens_coletados: Array[String] = []
```

4. Adicione um campo exportado para o último checkpoint alcançado:

```
@export var ultimo_checkpoint: String = ""
```

5. Salve o script (**Ctrl+S**).

6. Não crie uma instância `.tres` manual de `SaveData` em `res://` — diferente de `ItemData`, esta classe será instanciada em tempo de execução pelo `SaveComponent`, na Parte 2.

Resultado esperado

O projeto tem uma classe `SaveData` (Resource) com campos para itens coletados e checkpoint ativo, pronta para ser preenchida e gravada em tempo de execução.

Verificando

1. Confirme que `save_data.gd` está salvo em `scripts/resources/`, ao lado de `item_data.gd`.
2. Confirme que `class_name SaveData` não gera nenhum erro no painel de saída do editor.

Problemas comuns

- Criar `SaveData` como `Node` em vez de `Resource`: reforçar que apenas Resources podem ser serializados com `ResourceSaver` / `ResourceLoader`.
- Tentar salvar uma instância `.tres` de `SaveData` manualmente em `res://`, confundindo com o fluxo do `ItemData`: reforçar que dados de progresso do jogador não pertencem ao projeto (`res://`, somente leitura em builds exportados), e sim à pasta do usuário (`user://`), preenchida em tempo de execução.

Boas práticas

- Manter `SaveData` enxuto: apenas os dados que realmente precisam sobreviver ao fechamento do jogo, nunca referências diretas a Nodes da cena.
- Nomear os campos de forma que o propósito fique claro sem precisar consultar o script (`itens_coletados`, não `dados1`).

Comparação com Unity

A Unity não tem um Resource serializável nativo equivalente para este caso: o padrão mais comum é `PlayerPrefs` para dados simples (chave-valor) ou uma classe própria serializada manualmente em JSON/binário para dados estruturados como este. O Godot oferece um caminho mais direto — o próprio `Resource` já é serializável — sem exigir que a equipe defina um formato de arquivo próprio antes de começar a salvar dados.

Parte 2 — SaveComponent: gravando e lendo o SaveData em `user://`

Objetivo

Criar o `SaveComponent`, responsável por gravar e ler o `SaveData` na pasta `user://`.

Conceito

Se cada Scene que precisasse salvar progresso implementasse sua própria lógica de leitura e escrita em disco, o projeto acumularia lógica de serialização duplicada — exatamente o problema que a Rubrica 4 (Code Review) penaliza. O `SaveComponent` centraliza essa responsabilidade em um único lugar: qualquer Scene que precise salvar ou carregar progresso (o `Checkpoint` desta semana, e futuramente outros pontos do jogo) conversa apenas com o `SaveComponent`, nunca diretamente com `ResourceSaver / ResourceLoader` ou `FileAccess`.

O Godot oferece dois caminhos para gravar um Resource em disco:

`ResourceSaver.save()` / `ResourceLoader.load()`, que serializam o próprio objeto `SaveData` no formato nativo do Godot; ou `FileAccess` bruto, escrevendo manualmente (por exemplo, como JSON), útil quando o arquivo precisa ser legível fora do motor. Este tutorial usa `ResourceSaver / ResourceLoader`, por ser o caminho mais direto para um Resource já definido como `SaveData`.

Passo a passo

1. No FileSystem Dock, dentro de `scripts/components/`, crie um novo script chamado `save_component.gd`.
2. Declare a classe como `Node`, seguindo o mesmo padrão de Component da disciplina:

```
class_name SaveComponent
extends Node
```

3. Declare uma constante com o caminho do arquivo de save em `user://`:

```
const CAMINHO_SAVE := "user://save_data.tres"
```

4. Crie uma função `salvar()` que monta um `SaveData` a partir do estado atual do jogo (por enquanto, recebendo os itens coletados como parâmetro) e grava com `ResourceSaver`:

```
func salvar(itens_coletados: Array[String], checkpoint: String) -> void:
    var dados := SaveData.new()
    dados.itens_coletados = itens_coletados
    dados.ultimo_checkpoint = checkpoint
    ResourceSaver.save(dados, CAMINHO_SAVE)
```

5. Crie uma função `carregar()` que lê o arquivo, se existir, e devolve o `SaveData`:

```
func carregar() -> SaveData:
    if not FileAccess.file_exists(CAMINHO_SAVE):
        return null
    return ResourceLoader.load(CAMINHO_SAVE) as SaveData
```

6. Salve o script (**Ctrl+S**).
7. No editor, adicione um Node filho `SaveComponent` à Scene `Player` (ou à Scene raiz do nível de teste, conforme a organização já adotada pelo grupo), atribuindo o script `save_component.gd`.
8. Para testar, chame `salvar()` a partir de um script temporário (por exemplo, um atalho de teclado de debug) passando uma lista simples de itens, execute a cena, e verifique que o arquivo foi criado.
9. Localize a pasta `user://` no seu sistema operacional usando **Projeto > Abrir Pasta de Dados do Usuário**, no menu do editor, e confirme visualmente a presença de `save_data.tres`.
10. Chame `carregar()` logo em seguida e use `print()` para confirmar, no painel de saída, que os dados lidos são os mesmos gravados.

Resultado esperado

O `SaveComponent` grava um `SaveData` em `user://save_data.tres` e o recupera corretamente, confirmando que os dados sobrevivem entre chamadas independentes de `carregar()` — inclusive após fechar e reabrir o editor/jogo.

Verificando

1. Após chamar `salvar()`, confirme via **Projeto > Abrir Pasta de Dados do Usuário** que o arquivo `save_data.tres` existe fisicamente em disco.
2. Feche e reabra o jogo (ou o editor), chame apenas `carregar()`, e confirme que os dados retornados são os mesmos gravados na execução anterior.
3. Apague manualmente o arquivo `save_data.tres` e confirme que `carregar()` retorna `null` sem gerar erro.

Problemas comuns

- Gravar em um caminho dentro de `res://` em vez de `user://`: reforçar que `res://` é o projeto, somente leitura em builds exportados, e `user://` é a pasta de dados do usuário, a única gravável em tempo de execução.
- Esquecer de tratar o caso em que o arquivo de save ainda não existe (primeira execução do jogo), causando erro ao tentar carregar: reforçar o uso de `FileAccess.file_exists()` antes de `ResourceLoader.load()`.
- Instanciar `SaveComponent` mais de uma vez na mesma árvore de cena, criando ambiguidade sobre qual instância é a fonte de verdade: reforçar que deve haver um único `SaveComponent` ativo por vez, assim como um único `SaveManager`.

Boas práticas

- Manter a constante do caminho do arquivo em um único lugar (`CAMINHO_SAVE`), nunca repetida como string literal em outros scripts.
- O `SaveComponent` deve conhecer apenas `SaveData`, nunca Nodes específicos da cena — quem monta os dados a salvar é responsabilidade de quem chama `salvar()`, não do Component.
- Nomear o Component como `SaveComponent`, seguindo a convenção de sufixo `Component` já usada na disciplina, conforme a seção 9 do `PROJECT_ARCHITECTURE.md`.

Comparação com Unity

A Unity resolveria o mesmo problema com `PlayerPrefs` (para dados simples, chave-valor) ou com uma classe própria serializada manualmente em JSON via `System.IO`, decisão inteiramente a cargo da equipe — não existe um caminho "de fábrica" para dados estruturados como este `SaveData`. O Godot, por outro lado, aproveita o próprio `Resource` já definido e sua serialização nativa via `ResourceSaver` / `ResourceLoader`, sem exigir a definição de um formato intermediário. O conceito universal — transformar estado em memória em dado persistido, e reconstruí-lo na abertura seguinte — é idêntico nas duas engines; muda apenas o quão direto é o caminho padrão.

Parte 3 — Pickup e Chest: aplicando o `ItemData` à coleta

Objetivo

Construir as Scenes `Pickup` e `Chest`, que concedem um `ItemData` ao jogo reutilizando o contrato `Interactable`, e conectar o resultado da coleta ao `SaveManager` por um handler único.

Conceito

`Door` e `Lever` (Semana 5) reagem à interação sem produzir nenhum dado — apenas mudam o próprio estado. `Pickup` e `Chest` são o primeiro caso de um objeto interativo que **entrega algo ao jogo**: um `ItemData` (Semana 6). Os dois reutilizam, sem alteração, o mesmo método `interact()` já detectado pelo `InteractionComponent` do Player; mas, em vez do Signal genérico `interacted`, declaram um Signal próprio que carrega o dado coletado — `item_collected(item: ItemData)`.

Quem decide o que fazer com o item **não** é a Scene coletável. É um handler único, ligado ao `SaveManager`, exatamente como o `Checkpoint` (Parte 4) não conhece `ResourceSaver`: `Pickup` / `Chest` só anunciam "este item foi coletado" e permanecem desacoplados de inventário e de save. Isso é o que permitirá, na Semana 10, trocar o destino do item (do `SaveManager` para o `InventoryComponent`) sem tocar em `Pickup` / `Chest`.

`Pickup` é de uso único: some ao ser coletado. `Chest` tem um estado `aberto`: concede o item na primeira interação e fica vazio. Nenhum dos dois grava o próprio estado em disco no Módulo 2 — para não duplicar um item ao recarregar um save, o handler ignora um item cujo `nome` já esteja na lista de coletados.

Passo a passo

Parte A — o `SaveManager` guarda a lista de itens coletados

1. Abra a Orchestration `save_manager.torch` (Semana 4). Se a variável de demonstração (`itens_coletados: int = 0`) ainda existir com esse nome, troque o tipo para uma lista de nomes; se o grupo usou outro nome no desafio da Semana 4, mantenha aquele intacto e apenas adicione o campo novo — implementação de referência:

```
var itens_coletados: Array[String] = []
```

2. Adicione uma **função pública** que registra um item **uma única vez** — implementação de referência:

```
func registrar_item(nome: String) -> void:  
    if nome in itens_coletados:  
        return  
    itens_coletados.append(nome)
```

No grafo: função `registrar_item(nome)` → nó **Branch** (`nome in itens_coletados`) que retorna cedo se verdadeiro → senão, nó **Append** em `itens_coletados`.

Parte B — a Scene `Pickup`

3. Em `scenes/interactables/`, crie `Pickup.tscn` com Node raiz `Area3D`, um `CollisionShape3D` filho e uma malha/marcador visual (asset do Mini Dungeon) — a mesma estrutura já usada por `Door`.
4. Crie `pickup.gd` em `scripts/` (ou a Orchestration correspondente em `orchestrations/`), com `class_name Pickup` e, logo abaixo:

```
signal item_collected(item: ItemData)  
@export var item: ItemData
```

5. Implemente `interact()` (o mesmo nome exigido pelo contrato) para emitir o Signal e remover a Scene:

```
func interact() -> void:  
    if item == null:  
        return
```

```
item_collected.emit(item)
queue_free()
```

Parte C — a Scene **Chest**

6. Dupliche `Pickup.tscn` como `Chest.tscn` (ou crie do zero com a mesma estrutura), trocando a malha por um baú.
7. Crie `chest.gd` com `class_name Chest`:

```
signal item_collected(item: ItemData)
@export var item: ItemData
var aberto := false
```

8. `interact()` concede o item apenas na primeira interação:

```
func interact() -> void:
    if aberto or item == null:
        return
    aberto = true
    item_collected.emit(item)
```

Parte D — o handler único de coleta

9. No script do nó raiz do nível (`level_exploration.tscn`) — ou no `GameManager` , conforme a organização do grupo —, crie a função que recebe o item e o registra:

```
func _ao_coletar_item(item: ItemData) -> void:
    SaveManager.registrar_item(item.nome)
```

10. Para cada instância de `Pickup / Chest` no nível, conecte o Signal `item_collected` a `_ao_coletar_item` — pela subaba **Node > Signals** do editor, ou por código no `_ready()` do nível.
11. Posicione ao menos um `Pickup` e um `Chest` no nível, atribuindo a cada um, no Inspector, um `.tres` de `ItemData` do conjunto do grupo (Semana 6).

Resultado esperado

Um `Pickup` some ao ser interagido e um `Chest` abre uma única vez; em ambos os casos, o nome do `ItemData` correspondente aparece em `SaveManager.itens_coletados` , sem duplicatas — e nem `pickup.gd` nem `chest.gd` conhecem o `SaveManager` .

Verificando

1. Aproxime o Player de um `Pickup` e interaja: o objeto desaparece e `print(SaveManager.itens_coletados)` mostra o nome do item.
2. Interaja com um `Chest` duas vezes: o item é registrado apenas na primeira; a segunda interação não altera a lista.
3. Colete o mesmo tipo de item de duas fontes distintas e confirme que `itens_coletados` não contém nomes repetidos.

Problemas comuns

- Colocar a lógica de "o que o item faz" dentro de `pickup.gd / chest.gd`: a Scene apenas emite o `ItemData` — a reação pertence ao handler.
- Chamar `SaveManager.registrar_item()` diretamente de dentro de `Pickup / Chest`: isso reacopla a Scene ao save; a ligação deve passar pelo Signal `item_collected` e pelo handler.
- Esquecer de conectar o Signal `item_collected` de alguma instância, fazendo a coleta "não fazer nada".
- Deixar o `Chest` conceder o item a cada interação por não checar `aberto` antes de emitir.

Boas práticas

- Usar exatamente `item_collected(item: ItemData)` como Signal, conforme a seção 9 do `PROJECT_ARCHITECTURE.md` — `Door / Lever` continuam com `interacted`, sem carga de dados.
- Reaproveitar a estrutura de Scene de `Door` para `Pickup / Chest`: mesma pasta `scenes/interactables/`, mesma convenção de nomes.
- Manter **um único** handler de coleta no projeto, nunca um por Scene.

Comparação com Unity

Na Unity, `Pickup / Chest` implementariam a mesma interface de interação já definida pelo projeto (equivalente ao contrato `Interactable`) e exporiam um campo do `ScriptableObject` de item (equivalente ao `ItemData`). O evento de coleta seria um `UnityEvent<ItemData>` ou um `Action<ItemData>` em C#, ligado a um gerenciador único — a mesma separação entre "objeto que concede" e "sistema que decide o destino". O conceito de um objeto interativo que entrega um dado sem conhecer para onde ele vai é idêntico nas duas engines.

Parte 4 — Checkpoint: aplicando o contrato Interactable à persistência

Objetivo

Construir a Scene `Checkpoint`, que implementa o contrato `Interactable` e aciona o `SaveComponent` ao ser alcançada.

Conceito

O `Checkpoint` não introduz um novo mecanismo de interação: ele reaproveita, sem alteração, o mesmo contrato `Interactable` (via `has_method` ou interface do Orchestrator) já usado por `Door` e `Lever` desde a Semana 5. A única diferença está na reação ao ser interagido — em vez de abrir uma porta ou acionar uma alavanca, o `Checkpoint` chama `salvar()` no `SaveComponent`. Esse reaproveitamento é o teste real do desacoplamento ensinado na disciplina: um novo tipo de objeto interativo não exige nenhuma alteração no `InteractionComponent` do Player nem no contrato em si, apenas uma nova Scene que implementa a mesma interface.

Passo a passo

1. No FileSystem Dock, dentro de `scenes/interactables/`, crie uma nova Scene chamada `Checkpoint.tscn`, com um Node raiz apropriado (por exemplo, `Area3D`, seguindo o mesmo padrão já usado por `Door` ou `Lever`).
2. Adicione um `CollisionShape3D` filho, definindo a área de detecção do checkpoint.
3. Adicione uma malha ou marcador visual simples (asset do Mini Dungeon) como filho, apenas para o checkpoint ser identificável no nível.
4. Crie o script `checkpoint.gd` em `scripts/`, ou, se o grupo optar por Orchestrator, a Orchestration correspondente em `orchestrations/`.
5. Implemente o método exigido pelo contrato `Interactable` (o mesmo nome já usado por `Door / Lever`, por exemplo `interact()`), seguindo o padrão de duck typing (`has_method`) já estabelecido na Semana 5.
6. Atribua ao Node raiz uma `String` exportada `id_checkpoint`, com um valor único por instância, e adicione cada instância ao grupo `checkpoints` (subaba **Node > Groups**).
7. Dentro de `interact()`:
 - grave `SaveManager.ultimo_checkpoint = id_checkpoint` (o ponto de respawn corrente, em memória);

- obtenha a referência ao `SaveComponent` (do Player ou do Autoload, conforme a organização do grupo) e chame `salvar()`, passando `SaveManager.itens_coletados` (a lista alimentada pelo handler da Parte 3) e `id_checkpoint`.
8. Posicione ao menos uma instância de `Checkpoint.tscn` no nível de teste do grupo.
 9. Execute o projeto, aproxime o Player do `Checkpoint` e acione a interação da mesma forma já usada para portas e alavancas.
 10. Confirme, via **Projeto > Abrir Pasta de Dados do Usuário**, que o arquivo de save foi atualizado com o identificador do checkpoint alcançado.

Resultado esperado

O `Checkpoint` reage à interação do Player exatamente como `Door` ou `Lever` reagiriam, mas sua ação é gravar o progresso atual via `SaveComponent` — sem nenhuma lógica de interação própria além da já existente no contrato `Interactable`.

Verificando

1. Confirme que `Checkpoint.tscn` implementa o mesmo método de interação já usado por `Door / Lever`, sem duplicar a detecção de proximidade do `InteractionComponent` do Player.
2. Interaja com o `Checkpoint` e confirme, no painel de saída ou na pasta `user://`, que o `SaveData` foi atualizado.
3. Fixe um item coletado antes do checkpoint, interaja com o `Checkpoint`, feche e reabra o jogo, e confirme (via `carregar()`) que o item coletado e o identificador do checkpoint persistiram.

Problemas comuns

- Reimplementar a detecção de proximidade ou o input de interação dentro do `Checkpoint`, em vez de reutilizar o `InteractionComponent` do Player e o contrato já existente: reforçar que o `Checkpoint` só precisa implementar a reação (`interact()`), nunca a detecção.
- Chamar `ResourceSaver / FileAccess` diretamente dentro de `checkpoint.gd`, ignorando o `SaveComponent`: reforçar que toda gravação passa exclusivamente pelo `SaveComponent`, mesmo que pareça mais rápido chamar `ResourceSaver` diretamente.
- Esquecer de atribuir um identificador único a cada instância de `Checkpoint`, fazendo com que múltiplos checkpoints sobrescrevam o mesmo valor de forma indistinguível: reforçar a necessidade do campo `id_checkpoint`.

Boas práticas

- Tratar `Checkpoint` como qualquer outro objeto do contrato `Interactable` — mesma convenção de nomenclatura (`Checkpoint.tscn` , `checkpoint.gd`) e mesma pasta `scenes/interactables/` de `Door` , `Lever` , `Chest` , conforme a seção 8 do `PROJECT_ARCHITECTURE.md`.
- Dar feedback visual ou sonoro simples ao alcançar um checkpoint (mesmo que temporário, a ser substituído por áudio no Módulo 4), para o playtest do Encontro 2 já ser compreensível sem explicação verbal.

Comparação com Unity

Na Unity, o mesmo resultado exigiria uma classe que implementasse a interface de interação já definida pelo projeto (equivalente ao contrato `Interactable`), disparando a gravação via `PlayerPrefs` ou a serialização própria da equipe ao ser acionada por um Trigger Collider — estrutura equivalente à `Area3D` + `CollisionShape3D` do Godot. O conceito de reaproveitar o mesmo contrato de interação para um novo tipo de objeto, sem duplicar a lógica de detecção, é idêntico nas duas engines.

Parte 5 — Fechando o ciclo: carregar o save e escolher o ponto de spawn

Objetivo

Ligar as pontas soltas: carregar o `SaveData` quando o nível abre e fazer o `GameManager.spawn_player()` (Semana 4) escolher entre o `PlayerStart` e o último `Checkpoint` alcançado.

Conceito

Até agora o save é gravado (Parte 4) mas nunca **lido de volta** ao iniciar o jogo — e o `id_checkpoint` guardado não muda nada no mundo. Esta parte fecha o ciclo: ao carregar o nível, o script raiz pede o `SaveData` ao `SaveComponent` , copia os campos para o `SaveManager` (a fonte de verdade em memória) e chama `spawn_player()` . O `GameManager` então resolve `SaveManager.ultimo_checkpoint` (um id) para uma posição real, varrendo o grupo

`checkpoints` ; se não houver checkpoint ativo, usa o `PlayerStart` . É o mesmo `spawn_player()` da Semana 4 — só ganhou a escolha entre dois destinos, sem mudar quem o chama nem onde as coordenadas vivem.

O carregamento aqui cobre apenas `itens_coletados` e `ultimo_checkpoint` (o schema da Semana 7). Ampliar o `SaveData` para inventário real e vida é assunto de semanas seguintes.

Passo a passo

1. Abra a Orchestration `game_manager.torch` e amplie a função `spawn_player()` para considerar o checkpoint ativo antes do `PlayerStart` — implementação de referência:

```
func spawn_player() -> void:
    var player := get_tree().get_first_node_in_group("player")
    if player == null:
        return
    var destino: Node3D =
get_tree().get_first_node_in_group("player_start")
    if SaveManager.ultimo_checkpoint != "":
        for cp in get_tree().get_nodes_in_group("checkpoints"):
            if cp.id_checkpoint == SaveManager.ultimo_checkpoint:
                destino = cp
                break
    if destino:
        player.global_position = destino.global_position
```

2. No script ou Orchestration raiz de `level_exploration.tscn` , antes de chamar `spawn_player()` , carregue o save e popule o `SaveManager` — implementação de referência:

```
func _ready() -> void:
    var dados := $Player/SaveComponent.carregar() # ajuste o
caminho ao seu projeto
    if dados:
        SaveManager.itens_coletados = dados.itens_coletados
        SaveManager.ultimo_checkpoint = dados.ultimo_checkpoint
        GameManager.spawn_player()
```

3. Rode o nível sem nenhum save existente (apague `user://save_data.tres` se necessário) e confirme que o Player nasce no `PlayerStart`.
4. Alcance um `Checkpoint`, feche o jogo, reabra e confirme que o Player agora nasce **na posição daquele checkpoint**, não no `PlayerStart`.
5. Colete um item antes de outro checkpoint, alcance-o, feche/reabra e confirme que `SaveManager.itens_coletados` volta preenchido e o spawn é no checkpoint mais recente.

Resultado esperado

Reabrir o jogo recupera os itens coletados e faz o Player nascer no último `Checkpoint` alcançado; sem save, o Player nasce no `PlayerStart`. Nenhuma coordenada foi escrita no `GameManager` ou no `SaveManager`.

Verificando

1. Sem arquivo de save: Player no `PlayerStart`.
2. Com save após um checkpoint: Player na posição do `Checkpoint` de `id_checkpoint` igual a `ultimo_checkpoint`.
3. `game_manager.torch` e `save_manager.torch` não contêm nenhum `Vector3` / `Transform3D` — apenas ids e leitura de grupos.

Problemas comuns

- Ler o `SaveData` mas esquecer de copiar para o `SaveManager`, deixando `spawn_player()` sempre sem checkpoint ativo.
- Resolver o `id_checkpoint` para posição dentro do `SaveManager` em vez do `GameManager`: o `SaveManager` guarda o id; quem conhece a cena e faz a busca é o `GameManager`.
- Chamar `spawn_player()` antes de popular o `SaveManager` — a ordem no `_ready()` importa: carregar, copiar, depois spawnar.

Boas práticas

- Manter o `_ready()` do nível curto e explícito: carregar save → copiar para o `SaveManager` → `spawn_player()`. Essa sequência é o "ponto de entrada" do nível.
- Não tratar aqui o caso de morte do Player — `spawn_player()` é reutilizável e o fluxo de morte/respawn é assunto do Módulo 3.

Comparação com Unity

Na Unity, o equivalente seria um método `OnSceneLoaded` (ou `Start` de um gerenciador) que lê o save (via `PlayerPrefs` /JSON), guarda o id do checkpoint ativo em um singleton e reposiciona o Player no `Transform` do `SpawnPoint` ou do checkpoint correspondente. A lógica — carregar, resolver id para posição, mover o Player — é idêntica; muda apenas o mecanismo de leitura do save.

Ao final da semana

Este tutorial cobre apenas o Encontro 1. Ao final dele, o projeto do Vertical Slice deve conter:

- `SaveManager` (Autoload), contrato `Interactable`, Signals, `ItemData` /Enum e o conjunto de itens coletáveis das Semanas 4 a 6.
- O `GameManager` com `spawn_player()` ampliado para escolher entre `PlayerStart` e `Checkpoint` ativo.
- A classe `SaveData` (Resource), com campos para itens coletados e último checkpoint.
- O `SaveComponent`, centralizando a gravação e leitura do `SaveData` em `user://`.
- As Scenes `Pickup` e `Chest` em `scenes/interactables/`, cada uma com `@export var item: ItemData` e o Signal `item_collected(item: ItemData)`, além de um handler único que registra o item coletado em `SaveManager.itens_coletados`.
- Ao menos um `Pickup` e um `Chest` posicionados no nível, com itens do conjunto do grupo.
- Ao menos uma instância de `Checkpoint` (grupo `checkpoints`, `id_checkpoint` único), acionando o `SaveComponent` e gravando `SaveManager.ultimo_checkpoint`.
- O script raiz do nível carregando o `SaveData` no `_ready()` e chamando `GameManager.spawn_player()`.

Segundo o `PROJECT_ARCHITECTURE.md` (seção 6, Módulo 2), este resultado corresponde à conclusão dos itens "Pickup", "Chest", "SaveComponent / SaveData (Resource)", "Checkpoint" e "Carregar save ao iniciar + spawn_player() por checkpoint" do roadmap, preparando a integração final do Encontro 2.

Desafio

Não há desafio de solução livre neste encontro: a construção de `SaveData` / `SaveComponent` / `Pickup` / `Chest` / `Checkpoint` é guiada, servindo de base direta à integração avaliada do Encontro 2. A única variação por grupo é qual `ItemData` cada `Pickup` / `Chest` concede, a partir do conjunto próprio modelado na Semana 6.

Checklist

- ☐ Classe `SaveData` (Resource) criada, com campos `itens_coletados` e `ultimo_checkpoint`
- ☐ `SaveComponent` gravando e lendo `SaveData` corretamente em `user://`
- ☐ `SaveManager` com `itens_coletados: Array[String]` e `registrar_item()` idempotente
- ☐ Scenes `Pickup` e `Chest` implementando `interact()` e emitindo `item_collected(item: ItemData)`
- ☐ Handler único conectado ao Signal de cada `Pickup` / `Chest`, registrando o item no `SaveManager` (nenhuma Scene coletável conhece o `SaveManager`)
- ☐ Progresso confirmado como persistente entre execuções (fechar e reabrir o jogo)
- ☐ Scene `Checkpoint` implementando o contrato `Interactable`, sem lógica de interação duplicada
- ☐ Ao menos um `Checkpoint` posicionado e testado no nível, com `id_checkpoint` único e no grupo `checkpoints`
- ☐ `Checkpoint` grava `SaveManager.ultimo_checkpoint`; nível carrega o `SaveData` no `_ready()` e chama `GameManager.spawn_player()`
- ☐ `spawn_player()` posiciona o Player no `Checkpoint` ativo (ou no `PlayerStart` sem save) — nenhuma coordenada no `GameManager` / `SaveManager`
- ☐ Nenhuma gravação de save ocorrendo fora do `SaveComponent`

Glossário

- **Pickup**: Scene de item visível no mundo, de uso único; implementa `interact()`, expõe um `ItemData` e emite `item_collected` ao ser coletada, removendo-se em seguida.
- **Chest**: Scene de contêiner de uso único, com estado `aberto`; concede seu `ItemData` via `item_collected` apenas na primeira interação.
- **item_collected(item: ItemData)**: Signal próprio de `Pickup` / `Chest` que carrega o item coletado, consumido por um handler único ligado ao `SaveManager` (e, a partir da Semana 10, ao `InventoryComponent`).
- **SaveData**: Resource customizado que representa o estado de progresso do jogador a ser persistido em disco.

- **SaveComponent** : Component que centraliza a leitura e escrita do **SaveData** , evitando lógica de serialização duplicada entre Scenes.
- **user://** : pasta de dados do usuário, isolada do projeto (**res://**), gravável em tempo de execução mesmo em builds exportados.
- **ResourceSaver / ResourceLoader** : classes nativas do Godot para serializar e desserializar Resources em disco.
- **Checkpoint**: ponto de progresso do nível que aciona a gravação de estado ao ser alcançado, reutilizando o contrato **Interactable** , e grava seu **id_checkpoint** em **SaveManager.ultimo_checkpoint** .
- **spawn_player()** : método do **GameManager** (criado na Semana 4) que posiciona o Player ao carregar o nível — no **Checkpoint** ativo, se houver, ou no **PlayerStart** . As coordenadas vivem nesses Nodes, nunca no **GameManager** .
- **ultimo_checkpoint** : id do ponto de respawn corrente; guardado em **SaveManager** (memória) e em **SaveData** (disco), resolvido para posição pelo **GameManager** .

Referências

- Godot Documentation — Saving Games: https://docs.godotengine.org/en/stable/tutorials/io/saving_games.html
- Godot Documentation — Resources: <https://docs.godotengine.org/en/stable/tutorials/scripting/resources.html>
- Godot Documentation — File System (FileAccess): <https://docs.godotengine.org/en/stable/tutorials/scripting/filesystem.html>
- Orchestrator — Documentação oficial: <https://orchestrator.cratercrash.space/>
- Unity Manual (consulta comparativa) — PlayerPrefs: <https://docs.unity3d.com/Manual/class-PlayerPrefs.html>
- Canais recomendados para consulta complementar (não substituem a documentação oficial): GDQuest (<https://www.youtube.com/@GDQuest>), Clear Code (<https://www.youtube.com/@ClearCode>)